

Simulando o escalonamento preemptivo de processos que se comunicam usando pipe bidirecional

Primeiro Trabalho de Sistemas de Computação – INF 1316

- Prazo de entrega (envio por E-mail): 29 de setembro 2026, 23:59
- Data de apresentação (na bancada): 1 de outubro 2026, 11-13 hs

Introdução

O primeiro trabalho consiste em programar (em linguagem C e usando as primitivas fork/exec, sinais e comunicação Inter processos do Unix/Linux) um simulador de um kernel/núcleo de sistema operacional (**KernelSim**) que gerencia de **6 processos de aplicação** (A1, A2, ...A6) e intercala as suas execuções a depender se estão esperando pelo término de uma comunicação inter-rpcesso (IPC), ou seja, uma leitura ou escrita de/em pipes bidirecionais hipotéticos entre pares de processos consecutivo (A1<pipe>A2; A3<pipe>A4, A5<pipe>A6), ou uma interrupção sinalizando o termino de uma “fatia de tempo” (i.e. um *time slice*, ou intervalo de tempo para escalonamento RoundRobin). Para isso, você deverá implementar também um processo adicional que emula o controlador de interrupções (**InterContoler Sim**). Este deve gerar as interrupções do clock (referentes ao time-slice) e ao término da operação R/W nas pipes:

- a interrupção IRQ0 é do time-slice;
- a interrupção IRQ1 indica o término da operação de leitura em qualquer uma das três pipes; e
- IRQ2 indica o término da operação de escrita em qualquer uma das pipes.

Figura 1 mostra os elementos que devem fazer parte de seu sistema.

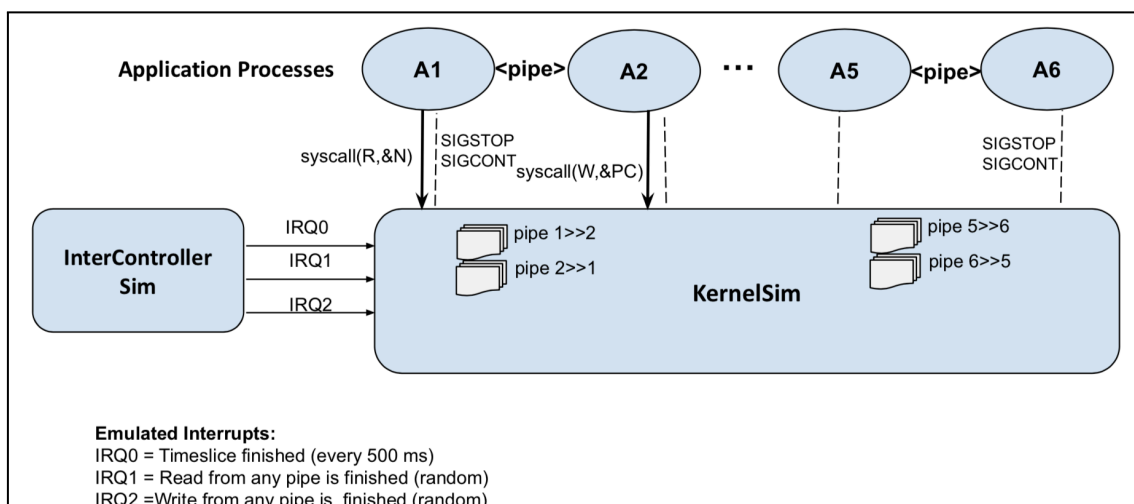


Figura 1: principais processos do seu sistema.

No seu programa, cada um dos elementos em azul-cinza da Fig. 1 deve ser um processo do Unix. Tanto o **InterController Sim** quanto o **KernelSim** devem ser processos infinitos que executam em paralelo.

Uso de Sinais UNIX

Para emular as interrupções (geradas por InterController Sim) e tratadas por KernelSim, você pode usar qualquer mecanismo de comunicação IPC, como pipes, Shmem ou FIFO, e deve usar os sinais SIGSTOP e SIGCONT para permitir que o KernelSim interrompa e retome, respectivamente, a execução dos processos de aplicação A1,..., A6.

Time-sharing

Qualquer processo Ax pode executar ininterruptamente, no máximo, por TimeSlice segundos. Portanto, quando chega um IRQ0 (Time Slice), KernelSim deve enviar um SIGSTOP para o processo que estava executando e escolher outro processo da aplicação Ay e novamente ativar o mesmo usando o sinal SIGCONT, a menos que este processo esteja esperando pelo retorno de uma chamada de sistema (syscall), **send()** ou **recv()**, na pipe com seu processo “vizinho da direita ou da esquerda”.

Mais ainda, se dois processos, digamos Ay e Az, tiverem executado uma syscall na pipe do mesmo tipo, digamos um send() em suas pipes (e a chamada de Ay aconteceu antes da chamada de Az), então o primeiro IRQ2 indicará o término do processamento pelo kernel, da chamada send() feita por Ay, que irá desbloquear Ay. E então o segundo IRQ2 sinalizará o término do segundo send(), de Az. (e irá desbloquear Az). O mesmo vale para a interrupção IRQ1 e para o efeito da chamada read() em qualquer número de processos.

Simulação de Pipes

Pipes são um mecanismo de comunicação interprocesso intermediado pelo kernel/núcleo, em que dois processos abrem um pipe e cada um deles pode escrever ou ler dados de um mesmo tipo. Tudo é feito por meio de chamadas de sistema que bloqueiam temporariamente o processo requisitante, como se fosse uma E/S. No nosso caso, todos os pipes serão implicitamente criados e cada pipe necessitará que o núcleo gerencie 2 buffers de inteiros (um para cada sentido). Então, para cada par de processos (A1 e A2), (A3 e A4) e (A5 e A6), será criado um pipe bidirecional, e o **kernel sim** terá que implementar 6 buffers (A1 >> A2, A1 << A2, A3 >> A4, A3 << A4, A5 >> A6, A5 << A6), que deverão guardar os dados do contador (PC) do processo remetente para o processo destinatário.

Chamada de Sistema (syscall)

Nesse trabalho, você vai emular uma system call **syscall(op, endereço)**, que, do ponto de vista de cada processo de aplicação, será realizada para enviar ou receber um dado do outro processo que faz dupla com ele. A1 <pipe1> A2; A3 <pipe2> A4, A5 <pipe3> A6. E cada processo terá duas variáveis **int PC, N**, onde PC é um contador local, e N deve conter o contador do “processo parceiro” recebido pelo pipe em comum.

Quando um processo de aplicação, digamos A1, executa send/recv, isso faz com que ele seja imediatamente interrompido pelo **KernelSim** por meio do sinal SIGSTOP e inserido na fila interna de processos bloqueados à espera do IRQ2 ou do IRQ1. Eventualmente, então, **InterController Sim** irá gerar uma nova interrupção IRQ2 ou IRQ1, que causará o desbloqueio do primeiro processo na fila (isso é que está na fila dos bloqueados por um send ou recv), e o **KernelSim** irá poder desbloquear o processo Unix A1 usando SIGCONT.

Em vez de simularmos um tempo concreto que cada syscall send() ou recv() demora, faremos o **InterControllerSim** gerar aleatoriamente interrupções para indicar o término de qualquer operação de acesso a pipe.

Para isso, implemente o **InterConntoller Sim** para gerar:

- Um IRQ0 (TimeSlice) a cada 500 ms (use sleep()) dentro do corpo do loop)
- Um IRQ1 com probabilidade $P_1 = 0.1$ (a cada 500 ms) /* probabilidade de um recv() ser concluído */
- Um IRQ2 com probabilidade $P_2 = 0.05$ (a cada 500 ms) /* probabilidade de um send() terminar */

Obs: Se você achar que essas frequências estão dificultando visualizar a execução, você pode alterar a periodicidade do Timslice e as probabilidades de P_1 e P_2 , mas sempre mantendo a probabilidade de P_1 maior do que a de P_2 . É como se fosse mais rápido ler dados do pipe do que escrever dados no pipe. Afinal, na escrita, talvez seja preciso alocar memória.

Processos de Aplicação

Cada processo de aplicação Ax deve consistir em um *loop* (de até MAX iterações) e ter um contador de interações chamado de ProgramCounter (PC), valor esse que deve ser compartilhado com o processo parceiro usando a escrita (send()) no pipe.

No corpo do loop, deverá haver um sleep(1) e uma escolha randômica, com baixa probabilidade, de o processo de aplicação requerer um syscall de leitura ou escrita no pipe. E o que ocorre se um processo, digamos A3, executa recv() e por uma “desgraça infinita”, até agora não existe nenhum dado que seu processo parceiro, A4, tenha colocado no pipe comum? Bem, nesse caso, o IRQ1 realmente desbloqueia A3, mas o valor retornado é 0 (nulo), indicando que não havia dados do processo parceiro no pipe. Assim, de certa forma, estamos simulando o recebimento do pipe no modo NO_WAIT, ou seja, sem que haja dados a serem lidos. E se não houver nenhuma chamada de sistema em todos os seis processos de aplicação, e o Kernel Sim receber um IRQ1 ou IRQ2? Bem, nesse caso a interrupção é perdida e o kernel não faz nada.

Exemplo:

```
int PC, N;
PC = 1;
while (PC < MAX) {
    PC++;
    sleep(0.5);
    if (d = rand()%100 +1 < 15) { // generate a random syscall
        if (d % 2) {
            Op = R;    // recv() on pipe p/ to get counter of other
            Data = &N;
```

```

    }
    else {
        Op = W          // d % 2 == 0
        Data = &PC;     // send() no pipe to share its own PC
    }
    // generate syscall (Op,Data)
}
sleep(0.5);
}

```

Troca de Contexto

No seu sistema, a troca de contexto acontecerá a cada vez que houver o chaveamento de um processo de aplicação para um outro. E, no caso específico do seu programa, esse contexto deve ser o contador PC do processo e o valor corrente da variável N. Então, ao reativar um processo anteriormente interrompido, o contexto completo anteriormente salvo deve ser restaurado, em parte, para garantir que todos os processos de aplicação Ax executem exatamente MAX interações. Além disso, se a interrupção de um processo se deveu a um syscall, então os parâmetros do syscall (R/W e addr) também devem fazer parte do contexto salvo e restaurado do processo. Para efeito de teste, use um MAX entre 5000 a 10000

Visualizando os estados dos processos

Além disso, seu simulador (envolvendo todos os processos incluindo **InterController Sim**), deve poder ser interrompido e reiniciado do ponto em que parou através Ctrl+Z *PID_simulador*) e, quando estiver parado, deve mostrar as seguintes informações sobre todos os processos A1-A5:

- valor do PC do N
- em que estado está (estado pronto, ou bloqueado)
- se estiver bloqueado, em qual dispositivo e com qual operação
- se está executando (porque o **KernelSim** enviou um SIGCONT e ainda não um SIGSTOP para ele)
- desde o início da execução e até esse valor de PC, quantas vezes o processo AP acessou cada um dos dispositivos.
- e se estiver terminado

Observações Finais

Como acontece a avaliação?

O trabalho pode ser feito individualmente ou em dupla (1). Deverá ser enviado na Data de Entrega (vide acima) para os e-mails do professor e do monitor, e deverá ser apresentado e explicado na Data de Apresentação na aula de laboratório (necessariamente com a presença da dupla). Cada dois dias de atraso resulta em um desconto de 1 ponto na nota máxima.

O que deve ser enviado/entregue?

Devem ser entregues o código-fonte e um relatório indicando que programas fazem parte do seu trabalho, incluindo eventuais programas de teste. Essa explicação também será objeto de avaliação.

- 1) ATENÇÃO: Só lembrando que o uso de LLMs (Large Language Models) está terminantemente proibido. Ou melhor, se for detectado o uso, a nota pelo trabalho será zero. Boa sorte.**